Cover Sheet

Reconfigurable Computing (ECE 531)

Title: Design_Exercise - 3

By: Saroj Shah

Date: Apr 19, 2024

Reconfigurable Computing, Professor Dr. James R Moulic

Introduction:

Following "Design Exercise: 3" part 1st shows the process of ROM instruction -> Fetch -> decode -> extraction (through ALU). Whereas 2nd part shows the creation of an IP object utilizing part 1st and 3rd part shows the creation of IPIBD using the above IP object and IP catalog.

Materials: Computer and Basys-3.

Procedure:

1. For 1st part, in the Vivado program write code that depicts the ROM and ALU working and verify using Basys3
2. For the 2nd part, in Vivado, utilizing part 1st create the IP object.
3. For 3rd part, again in Vivado, by utilizing part 2nd and IP catalog create Block Design (IPIBD).
4. Now input your desired inputs (address) on Basys and observe and compare the result.

----------Part# 1st----------

Code and Simulation:

a) Part 1:

I. Code for Main VHDL module:

```
----------------------------------------------------------------------------------
-- Company: Student
-- Engineer: Saroj
--
-- Create Date: 04/13/2024 12:39:46 AM
-- Design Name: Creating_Flat and IP integrator
-- Module Name: design_EX3_1and2 - Behavioral
-- Project Name: Design HW 3
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;


entity design_EX3_1and2 is
    Port ( Addr : in std_logic_vector(3 downto 0);
           S: in std_logic;
           mux_out: out std_logic_vector(13 downto 0));
end design_EX3_1and2;
```

```
21  architecture Behavioral of design_EX3_land2 is
22  --Creating 16X14 bit ROM i.e, ROM which has 16 address
23  -- and at each address have the pre-defined 14 bit long value
24
25   type ROM_16x14 is array(0 to 15)of std_logic_vector(13 downto 0);
26   constant rom_array : ROM_16x14 :=(
27                                     0 => "00100010100101",
28                                     1 => "00101011011111",
29                                     2 => "01010011100101",
30                                     3 => "01010010100111",
31                                     4 => "11011011011111",
32                                     5 => "11101001101101",
33                                     6 => "11101101100011",
34                                     7 => "10000010111010",
35                                     8 => "10010010111010",
36                                     9 => "10100010111010",
37                                     10 => "10110010111010",
38                                     11 => "00000000000000",
39                                     12 => "00000000000000",
40                                     13 => "00000000000000",
41                                     14 => "00000000000000",
42                                     15 => "00000000000000",
43                                     others => "00000000000000");
44
45   -- Creating internal signal,
46      signal A_signal : unsigned(4 downto 0) := ("00000");
47      signal B_signal : unsigned(4 downto 0):= ("00000");
48      signal Opcode_signal : std_logic_vector(3 downto 0) := ("0000");
49      signal address :  integer range 0 to 15;
50
51      -- These signal are used to transfer data to, from and between RAM_process and ALU_process
52      signal data_out:std_logic_vector(13 downto 0):= ("00000000000000");
53      signal ALU_out : std_logic_vector(9 downto 0):=("0000000000");
54
```

```
55  begin
56
57  -- Reading provided address signal and implementing pre-stored memory instruction according to that address
58      ROM_process: process(Addr, data_out, address)
59          begin
60              address <= TO_INTEGER(unsigned(Addr));
61              data_out <= rom_array(address);
62              Opcode_signal <= data_out(13 downto 10);
63              A_signal <= unsigned (data_out(9 downto 5));
64              B_signal <= unsigned (data_out(4 downto 0));
65
66      end process ROM_process;
67      -------------------- ALU Function -----------
68      ALU_process: process(Opcode_signal,A_signal,B_signal)
69          begin
70              if(Opcode_signal = "0010")then                                          -- Result - Sum
71                  ALU_out <= std_logic_vector(("00000" & A_signal) + ("00000" & B_signal));
72
73               elsif (Opcode_signal = "0101")then                                     -- Result Difference
74                  ALU_out <= std_logic_vector(("00000" & A_signal) - ("00000" & B_signal));
75
76               elsif (Opcode_signal = "1101")then                                     -- Result - Multiplication
77                   ALU_out <= std_logic_vector(( A_signal) * ( B_signal));
78
79               elsif (Opcode_signal = "1110")then                                     --Result - Quotient and Remainder
80                  ALU_out <= std_logic_vector ((A_signal / B_signal) & (A_signal rem B_signal));
81
82               elsif (Opcode_signal = "1000") then                                    -- Result - gives logical AND operation
83                  ALU_out <= std_logic_vector(("00000" & A_signal) and ("00000" & B_signal));
84
85               elsif (Opcode_signal = "1001") then                                    -- Result - gives logical OR operation
86                  ALU_out <= std_logic_vector(("00000" & A_signal) or ("00000" & B_signal));
87
88               elsif (Opcode_signal = "1010") then                                    -- Result - gives logical XOR operation
89                  ALU_out <= std_logic_vector(("00000" & A_signal) xor ("00000" & B_signal));
90
```

```
                  _         _     _                 _                       _
90
91              elsif (Opcode_signal = "1011") then                                          -- Result - gives logical NOT  operation
92                  ALU_out <= std_logic_vector (not ("00000" &  A_signal));
93
94              elsif (Opcode_signal = "1100") then                                                                  -- For any other Opcode
95                  ALU_out <= "0000000000";
96
97              elsif (Opcode_signal = "1101") then
98                  ALU_out <= "0000000000";
99
100             elsif (Opcode_signal = "1110") then                                                                  -- For any other Opcode
101                 ALU_out <= "0000000000";
102
103             elsif (Opcode_signal = "1111") then                                                                  -- For any other Opcode
104                 ALU_out <= "0000000000";                                                                    -- For any other Opcode
105
106              else                                                                        -- For any other Opcode
107                 ALU_out <= "UUUUUUUUUU";
108             end if;
109     end process ALU_process;
110 ------------ Output Process--------------------------
111     Output_Process: process(S, ALU_out, data_out)
112       begin
113 --
114         if (S = '0') then                                -- when S = 1 memory output the result of ALU
115             mux_out <= data_out;
116         else
117             mux_out <=  ("0000" & ALU_out);                          -- when S = 0 memory output the instrucion input
118         end if;
119
120     end process Output_Process;
121
122 end Behavioral;
```

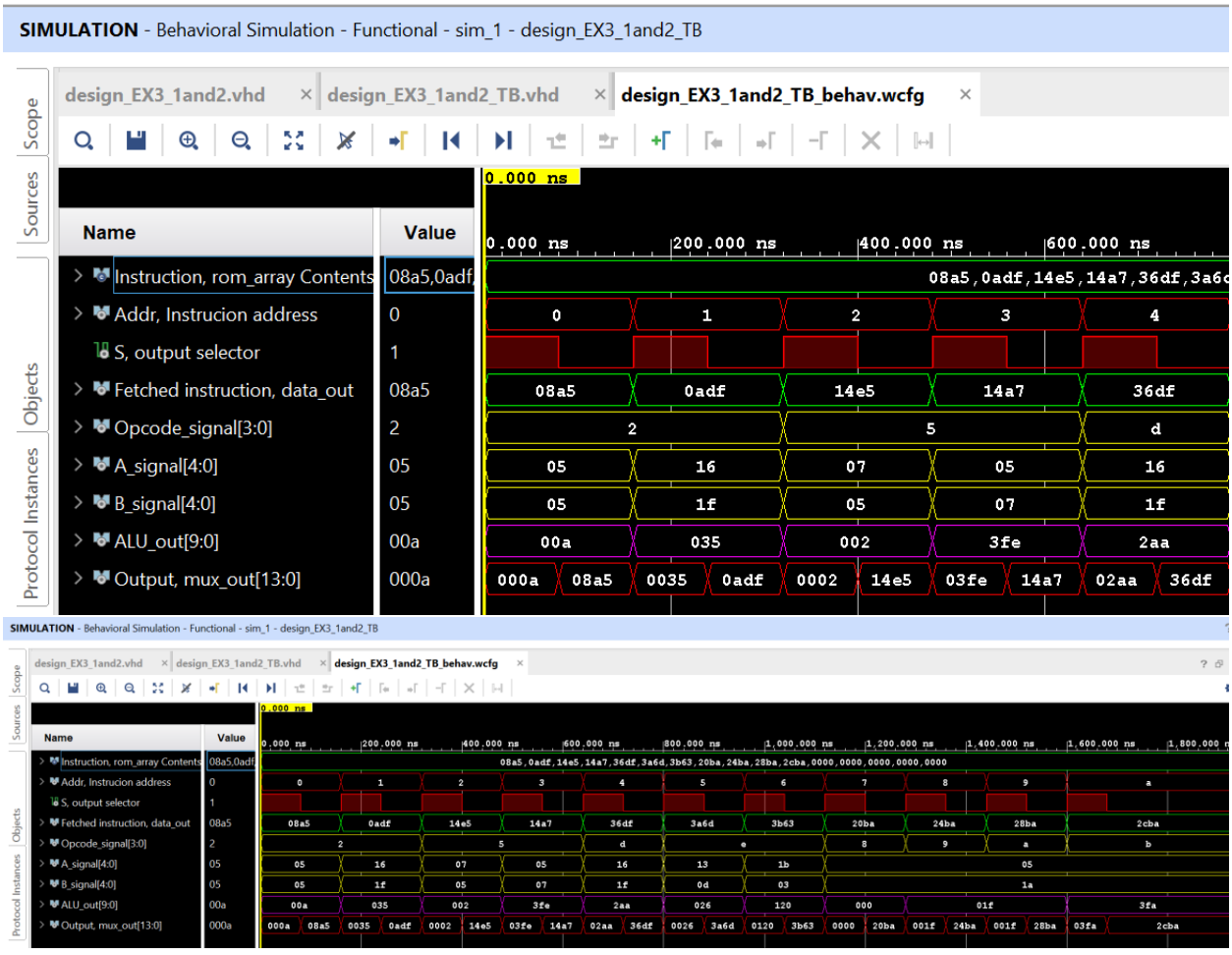
II. Code for Testbench:

```
9   library IEEE;
10  use IEEE.STD_LOGIC_1164.ALL;
11  use IEEE.NUMERIC_STD.ALL;
12
13  entity design_EX3_1and2_TB is
14  --  Port ( );
15  end design_EX3_1and2_TB;
16
17  architecture bench of design_EX3_1and2_TB is
18
19    component design_EX3_1and2
20        Port ( Addr : in std_logic_vector(3 downto 0);
21               S: in std_logic;
22               mux_out: out std_logic_vector(13 downto 0));
23    end component;
24
25    signal Addr: std_logic_vector(3 downto 0);
26    signal S: std_logic;
27    signal mux_out: std_logic_vector(13 downto 0);
28
29  begin
30  uut: design_EX3_1and2 port map ( Addr    => Addr,
31                                   S       => S,
32                                   mux_out => mux_out );
33
34    stimulus: process
35    begin
```

```
begin
  -- Put initialisation code here
    Addr <= "0000";
    S <= '1';
    wait for 80ns;
    Addr <= "0000";
    S <= '0';
    wait for 80ns;

    Addr <= "0001";
    S <= '1';
    wait for 80ns;
    Addr <= "0001";
    S <= '0';
    wait for 80ns;

    Addr <= "0010";
    S <= '1';
    wait for 80ns;
    Addr <= "0010";
    S <= '0';
    wait for 80ns;

    Addr <= "0011";
    S <= '1';
    wait for 80ns;
    Addr <= "0011";
    S <= '0';
    wait for 80ns;

    Addr <= "0100";
    S <= '1';
    wait for 80ns;
    Addr <= "0100";
    S <= '0';
    wait for 80ns;


    Addr <= "0101";
    S <= '1';
    wait for 80ns;
    Addr <= "0101";
    S <= '0';
    wait for 80ns;

    Addr <= "0110";
    S <= '1';
    wait for 80ns;
    Addr <= "0110";
    S <= '0';
    wait for 80ns;

    Addr <= "0111";
    S <= '1';
    wait for 80ns;
    Addr <= "0111";
    S <= '0';
    wait for 80ns;

    Addr <= "1000";
    S <= '1';
    wait for 80ns;
    Addr <= "1000";
    S <= '0';
    wait for 80ns;

    Addr <= "1001";
    S <= '1';
    wait for 80ns;
    Addr <= "1001";
    S <= '0';
    wait for 80ns;
```

```
106
107     Addr <= "1010";
108     S <= '1';
109     wait for 80ns;
110     Addr <= "1010";
111     S <= '0';
112     wait for 80ns;
113
114     -- Put test bench stimulus code here
115     wait;
116   end process;
117
118 end bench;
```

b) Timing Diagram:

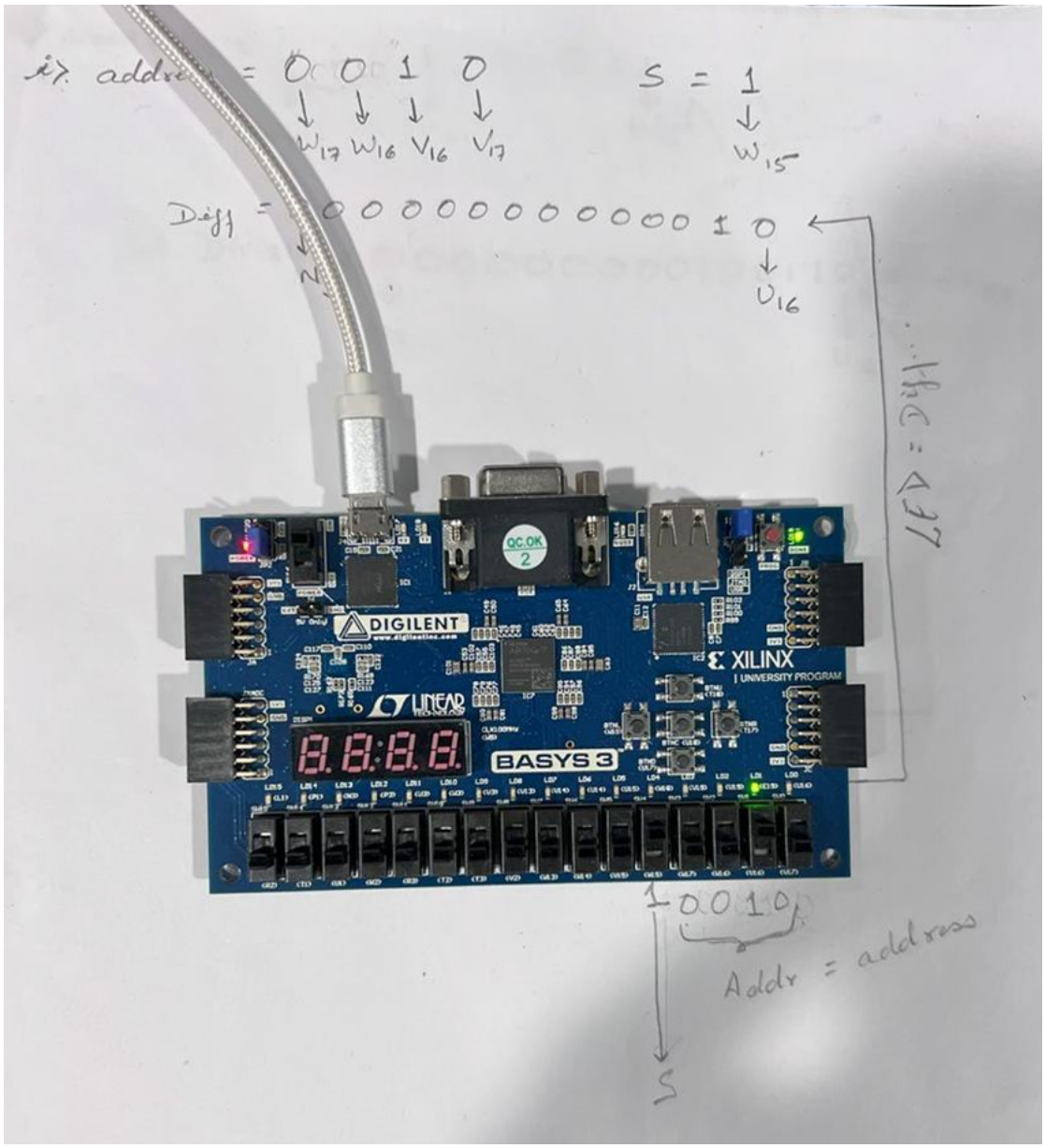


Testing and Results:

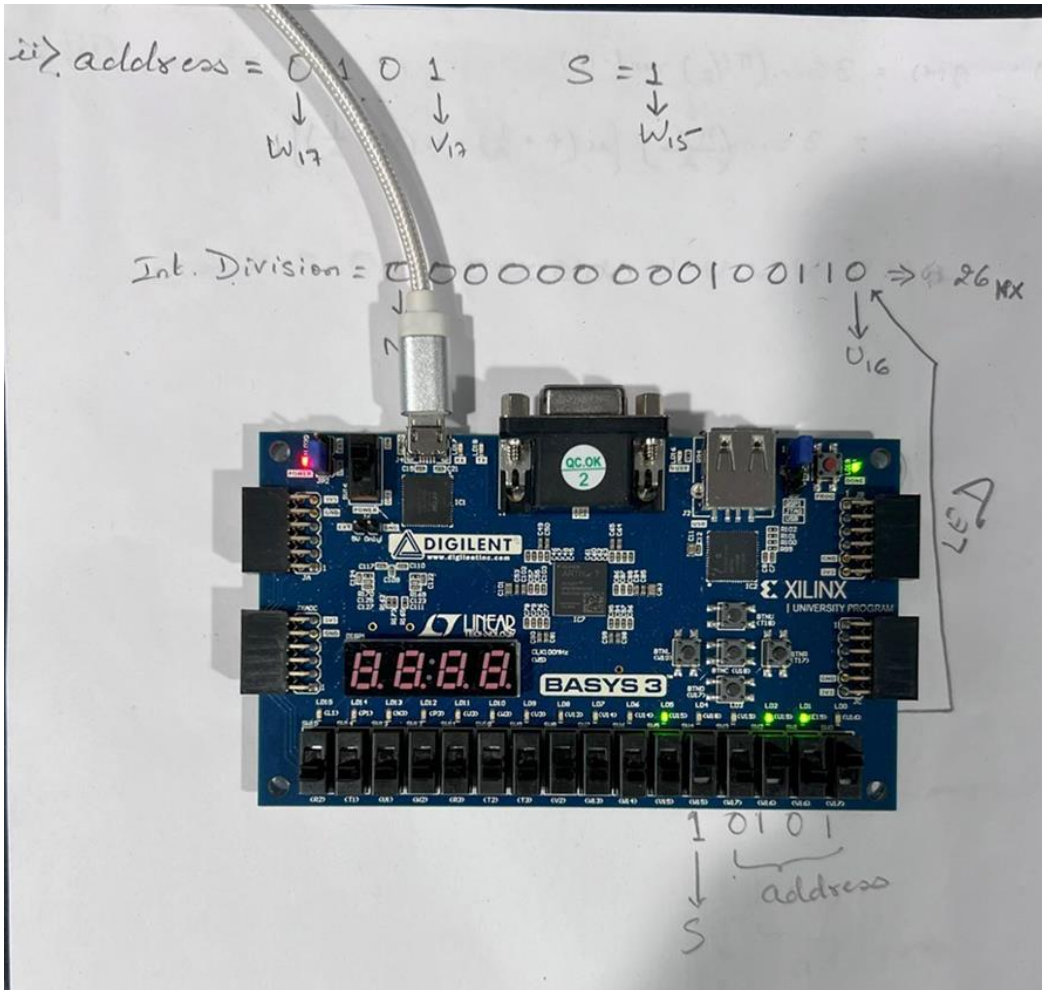
Please follow the picture. The picture shows the test for given inputs and the results received from it.

c) Photos running my design:

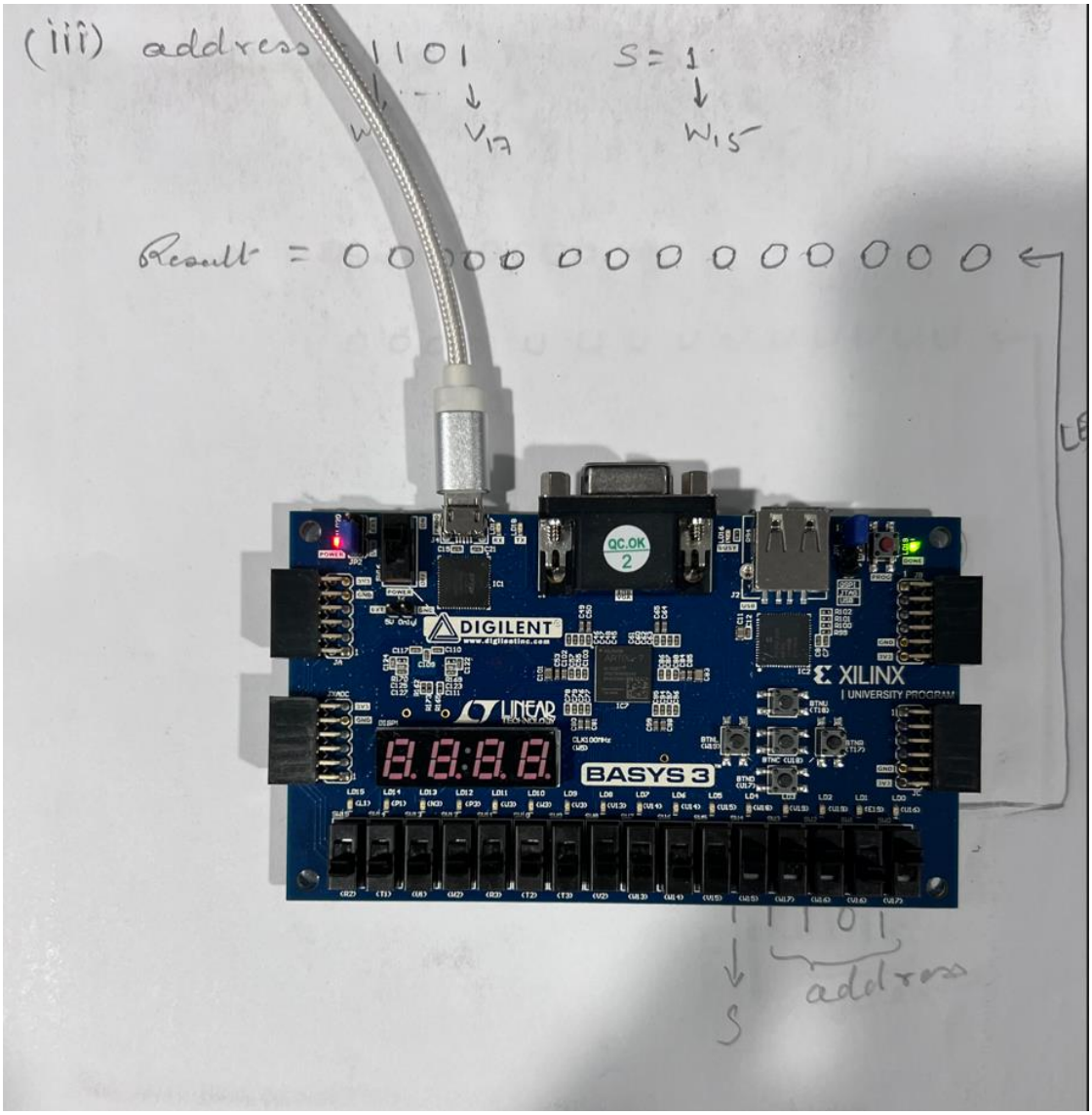
I. **address “0010”; S = ‘1’**



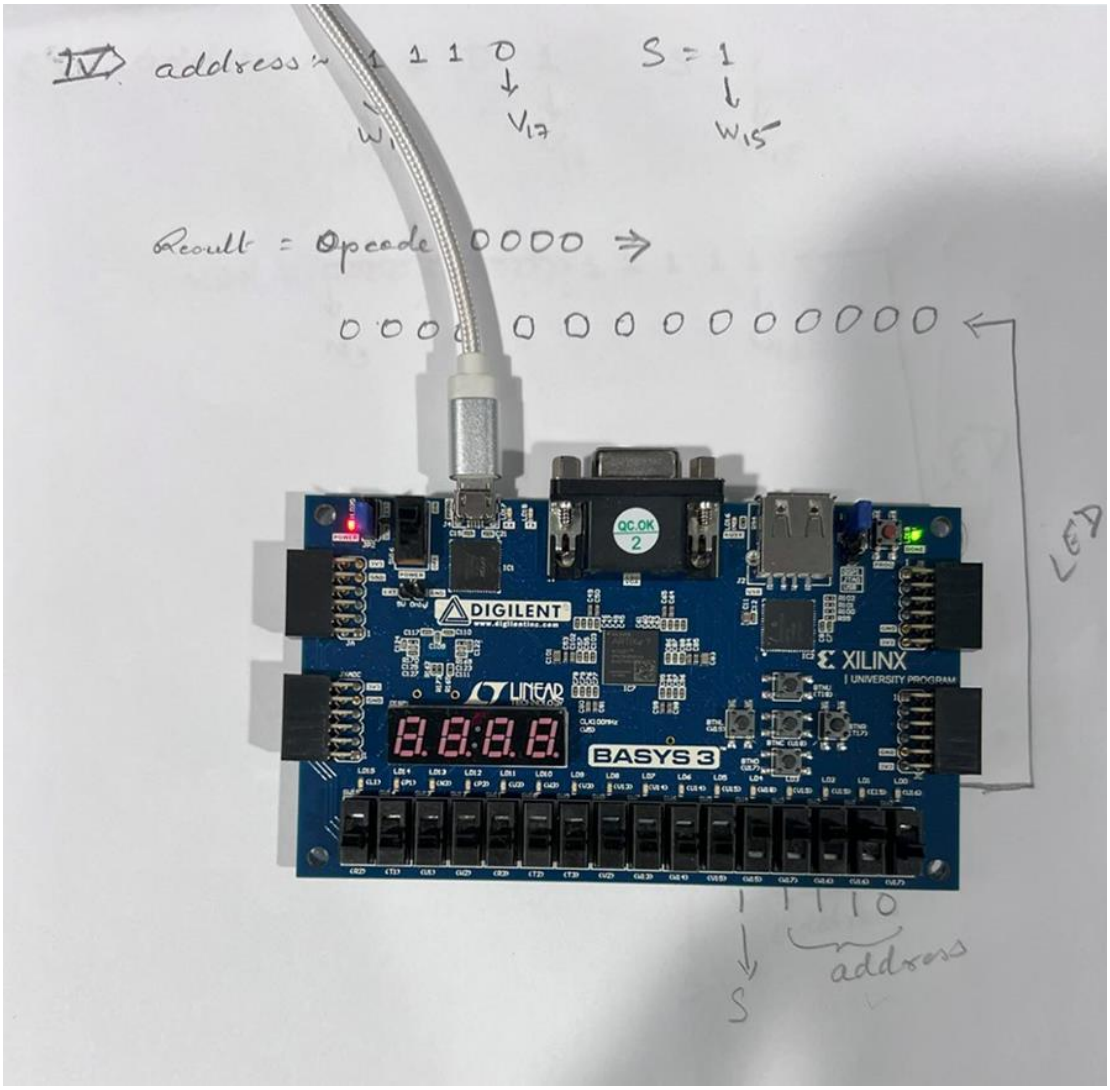
II. **address “0101” ; S = ‘1’**



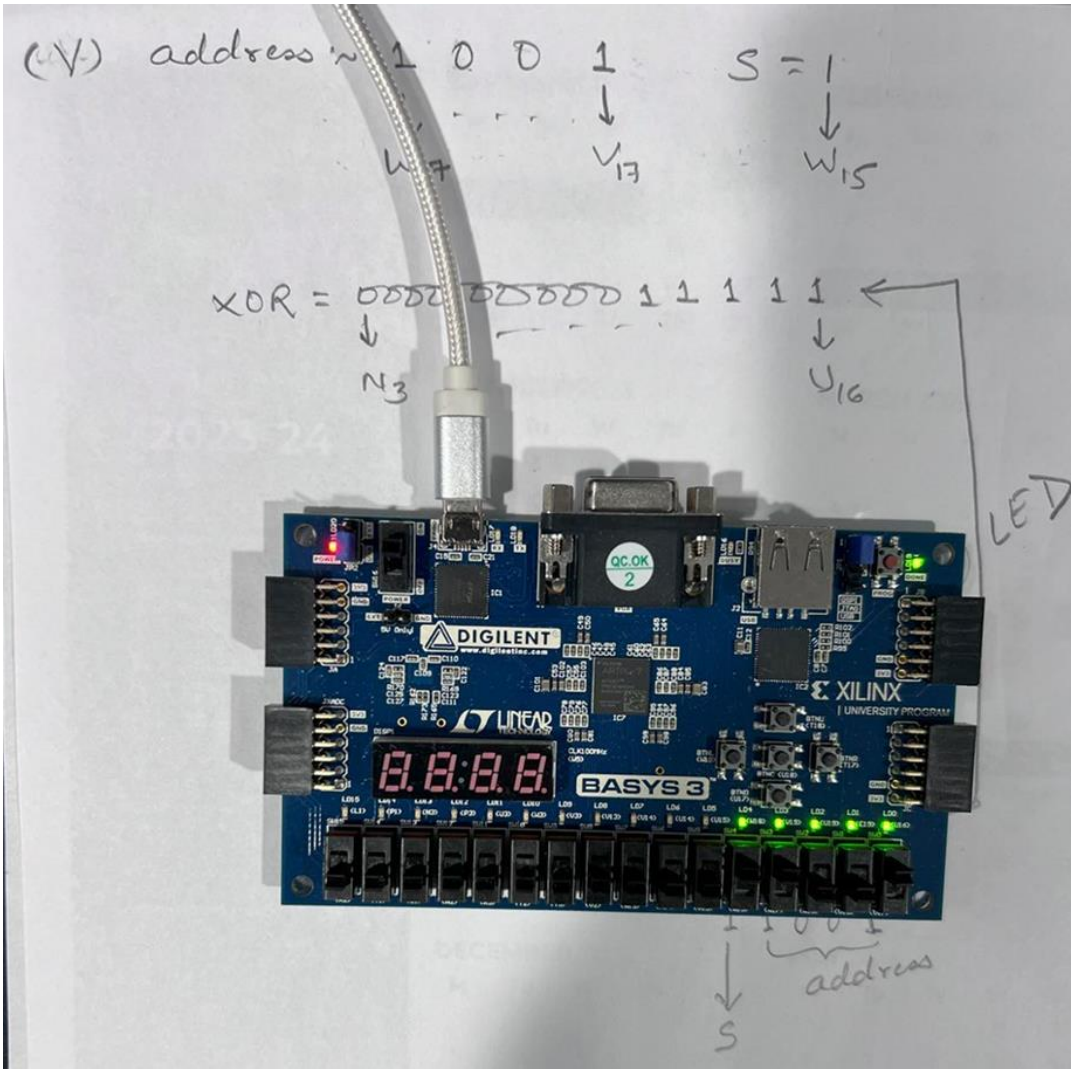
III. **address “1101”; S = ‘1’**



IV. **address “1110”; S = ‘1’**



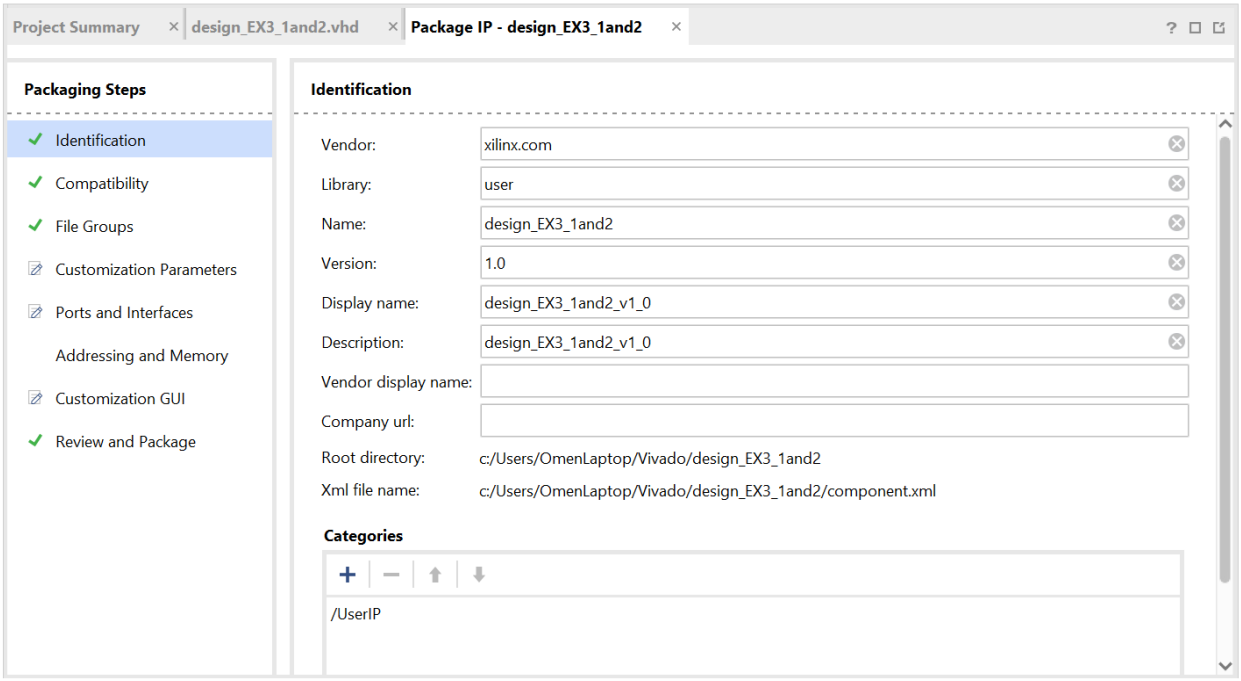
V. **address “1001”; S = ‘1’**



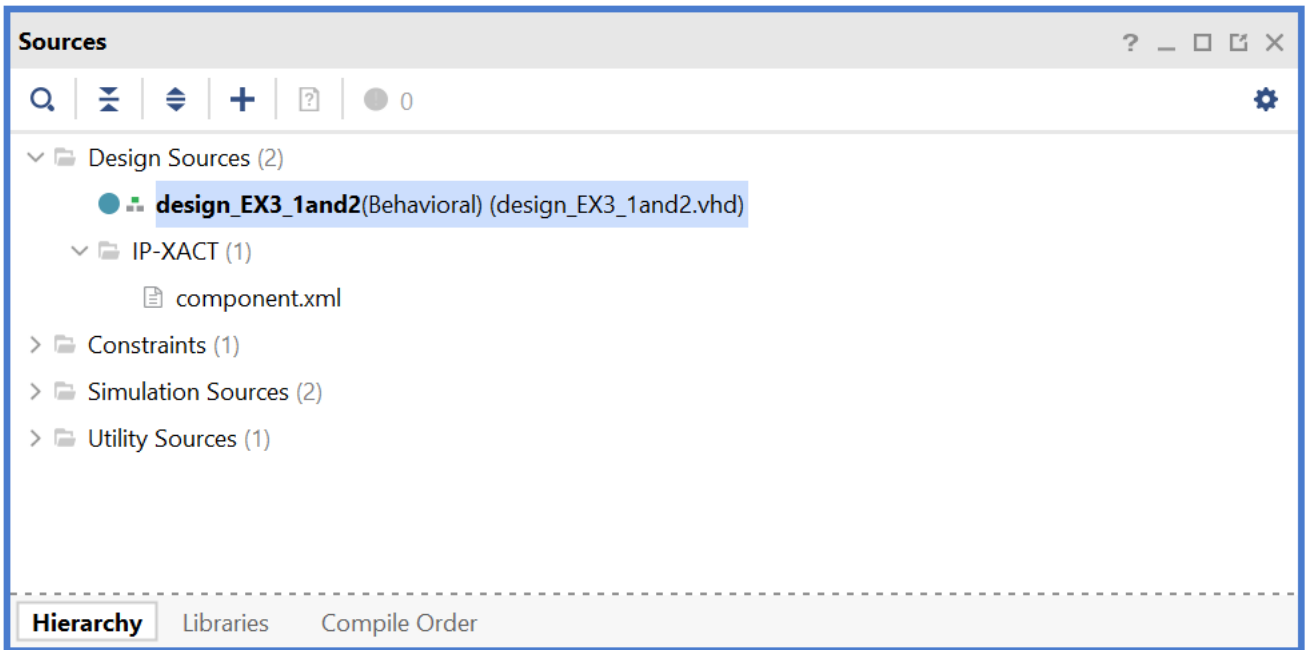
----------Part# 2nd----------

For the User IP part:

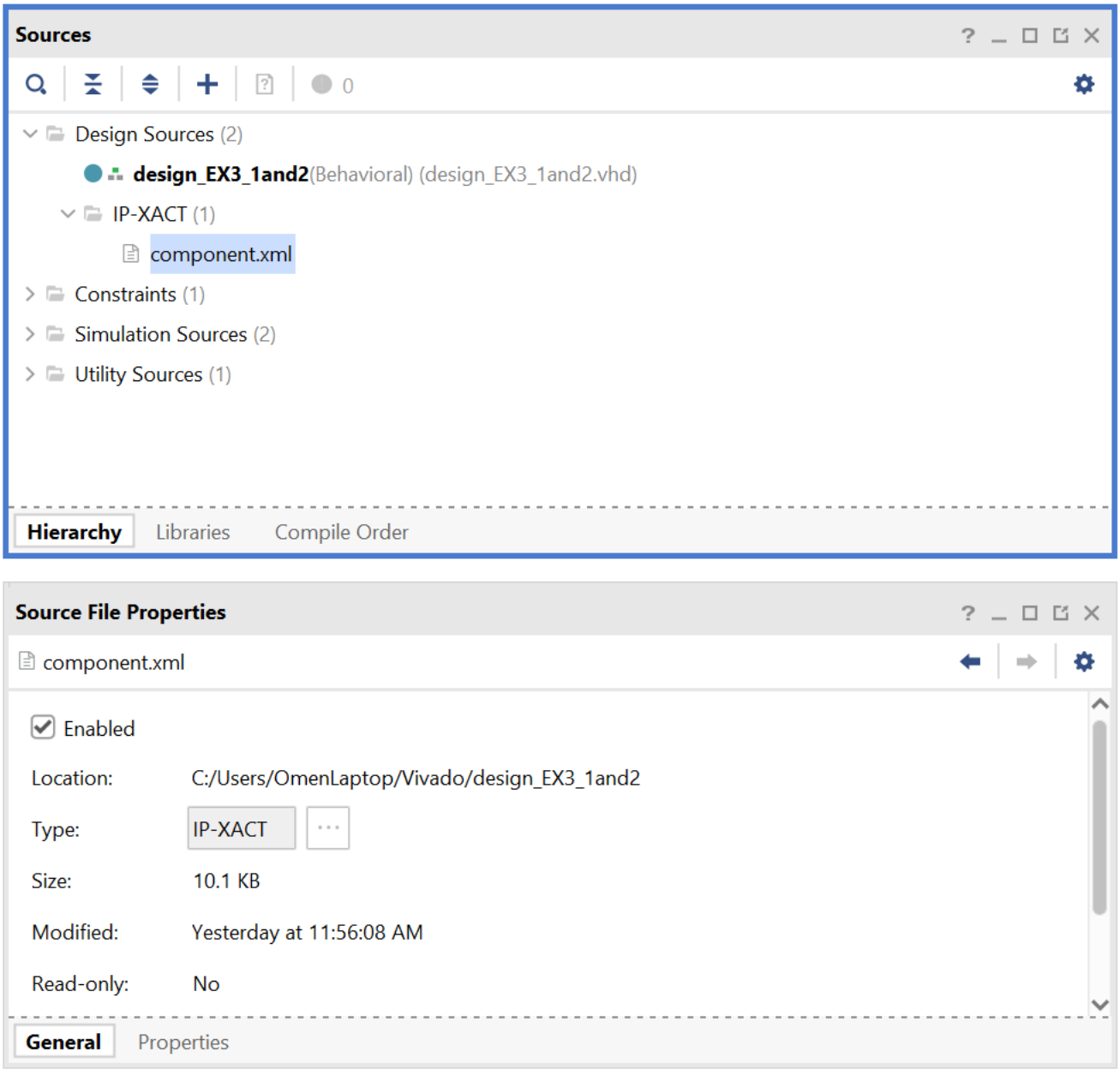
a) Image/photo of the User IP Packager Settings Table from the "Edit Packaged IP" Tab in the Vivado IDE's Flow Navigator).



b) Image/photo of the project source file and file properties panels in the Vivado IDE. Photos must clearly show the project VHDL source file name and saved location.



c) Image/photo of the project IP-XACT source file and file properties panels in the Vivado IDE. Photos must clearly show the component XML source file name and saved location.



----------Part# 3rd----------

IPIBD part:

a) Photos/images of the HDL wrapper code listing and testbench source codes from the IPIBD project.

```
--Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
--Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.
----------------------------------------------------------------------------------
--Tool Version: Vivado v.2023.2 (win64) Build 4029153 Fri Oct 13 20:14:34 MDT 2023
--Date        : Wed Apr 17 12:32:29 2024
--Host        : OmenLaptop running 64-bit major release  (build 9200)
--Command     : generate_target design_IP_design3_Q3_wrapper.bd
--Design      : design_IP_design3_Q3_wrapper
--Purpose     : IP block netlist
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
library UNISIM;
use UNISIM.VCOMPONENTS.ALL;

entity design_IP_design3_Q3_wrapper is
  port (
    Addr_0 : in STD_LOGIC_VECTOR ( 3 downto 0 );
    S_0 : in STD_LOGIC;
    clka_0 : in STD_LOGIC;
    douta_0 : out STD_LOGIC_VECTOR ( 13 downto 0 );
    mux_out_0 : out STD_LOGIC_VECTOR ( 13 downto 0 )
  );
end design_IP_design3_Q3_wrapper;

architecture STRUCTURE of design_IP_design3_Q3_wrapper is
  component design_IP_design3_Q3 is
  port (
    Addr_0 : in STD_LOGIC_VECTOR ( 3 downto 0 );
    S_0 : in STD_LOGIC;
    mux_out_0 : out STD_LOGIC_VECTOR ( 13 downto 0 );
    clka_0 : in STD_LOGIC;
    douta_0 : out STD_LOGIC_VECTOR ( 13 downto 0 )
  );
  end component design_IP_design3_Q3;
```

```
begin
design_IP_design3_Q3_i: component design_IP_design3_Q3
     port map (
      Addr_0(3 downto 0) => Addr_0(3 downto 0),
      S_0 => S_0,
      clka_0 => clka_0,
      douta_0(13 downto 0) => douta_0(13 downto 0),
      mux_out_0(13 downto 0) => mux_out_0(13 downto 0)
    );
end STRUCTURE;
```

Testbench:

Sources | Source File Properties

Project Summary | design_IP_design3_Q3_wrapper.vhd | test_design_IP_design3_Q3.vhd

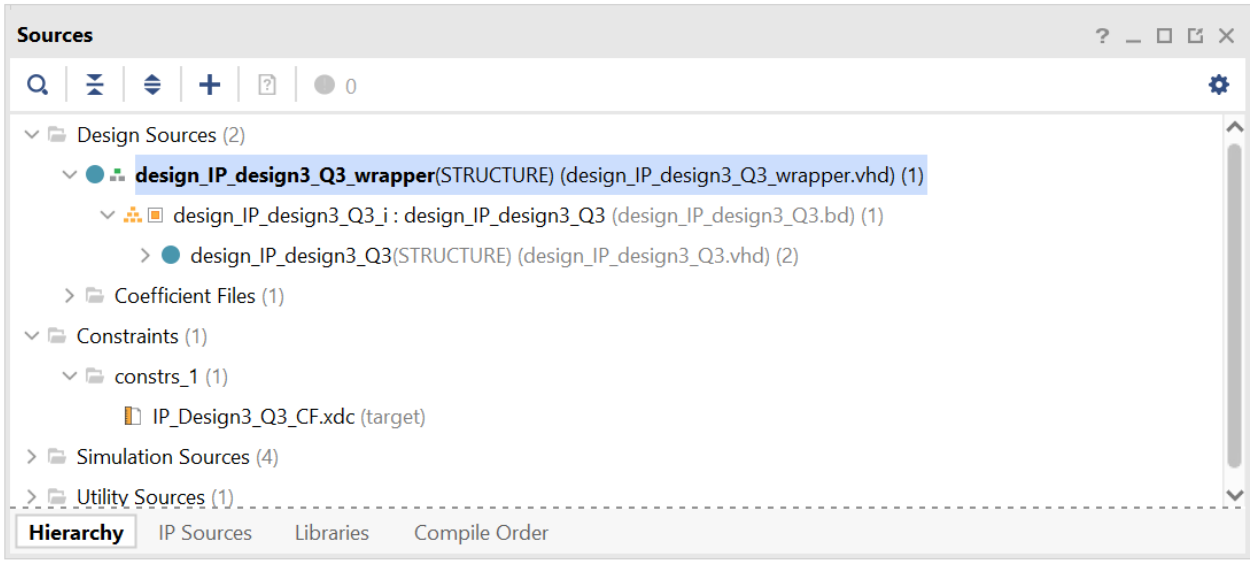
C:/Users/OmenLaptop/Vivado/IP_design3_Q3/IP_design3_Q3.srcs/sim_1/imports/Vivado/test_design_IP_design3_Q3.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity test_design_IP_design3_Q3 is
5  end test_design_IP_design3_Q3;
6
7
8  architecture TB of test_design_IP_design3_Q3 is
9
10 component design_IP_design3_Q3 is
11 port (
12   Addr_0 : in STD_LOGIC_VECTOR (3 downto 0);
13   S_0 : in STD_LOGIC;
14   clka_0 : in STD_LOGIC;
15   douta_0 : out STD_LOGIC_VECTOR (13 downto 0);
16   mux_out_0 : out STD_LOGIC_VECTOR (13 downto 0)
17 );
18 end component design_IP_design3_Q3;
19
20 signal Addr_0 : STD_LOGIC_VECTOR (3 downto 0);
21 signal S_0 : STD_LOGIC;
22 signal clka_0 : STD_LOGIC;
23 signal douta_0 : STD_LOGIC_VECTOR (13 downto 0);
24 signal mux_out_0 : STD_LOGIC_VECTOR (13 downto 0);
25 constant delay : time := 80 ns;
26
27 begin
28
29 DUT: component design_IP_design3_Q3 port map (
30   Addr_0 => Addr_0,
31   S_0 => S_0,
32   clka_0 => clka_0,
33   douta_0 => douta_0,
34   mux_out_0 => mux_out_0
35 );
36
37 process
38 begin
39   clka_0 <= '0';
40   wait for 5.0 ns;
41   clka_0 <= '1';
42   wait for 5.0 ns;
43 end process;
44 Stimulus : process
45 begin
46
47     Addr_0 <= "0000";
48     S_0 <= '1';
49     wait for delay;
50     Addr_0 <= "0000";
51     S_0 <= '0';
52     wait for delay;
53
```

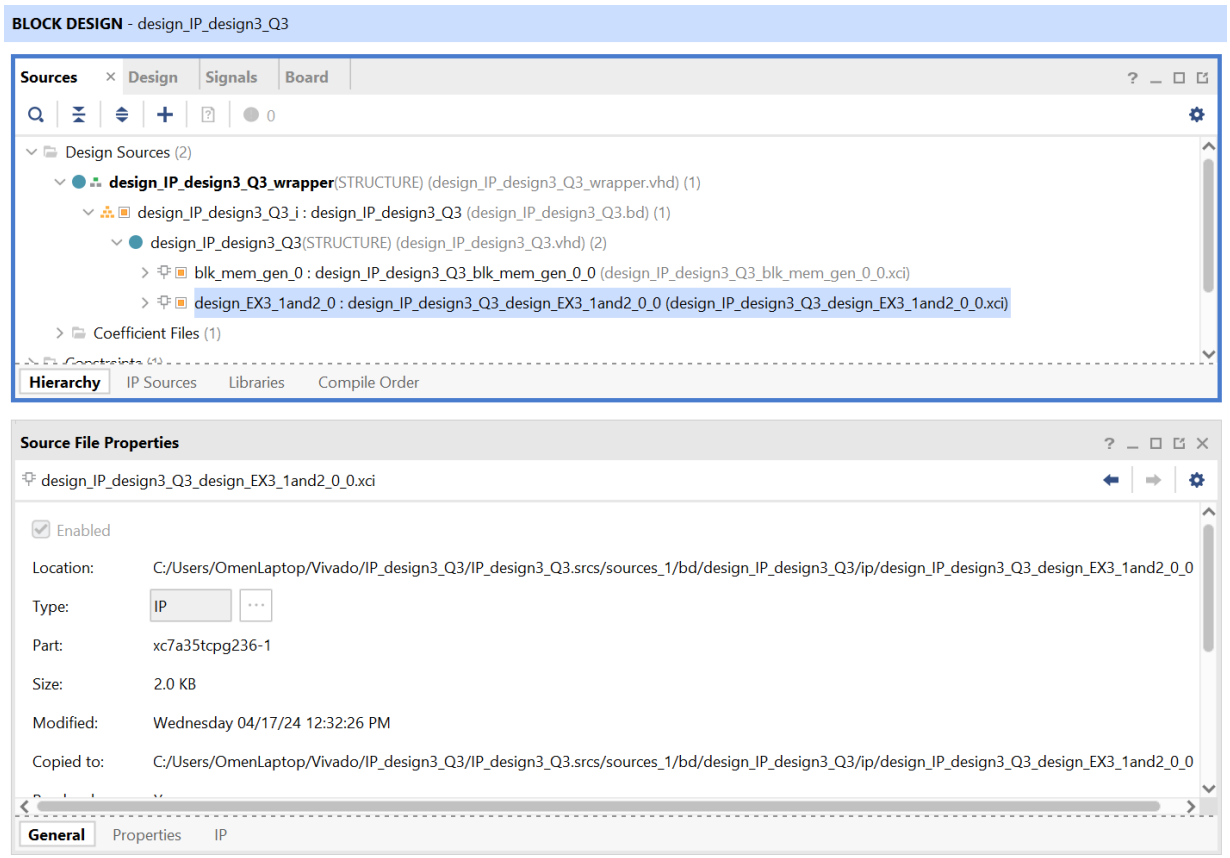
```
54      Addr_0 <= "0001";
55      S_0 <= '1';
56      wait for delay;
57      Addr_0 <= "0001";
58      S_0 <= '0';
59      wait for delay;
60
61      Addr_0 <= "0010";
62      S_0 <= '1';
63      wait for delay;
64      Addr_0 <= "0010";
65      S_0 <= '0';
66      wait for delay;
67
68      Addr_0 <= "0011";
69      S_0 <= '1';
70      wait for delay;
71      Addr_0 <= "0011";
72      S_0 <= '0';
73      wait for delay;
74
75      Addr_0 <= "0100";
76      S_0 <= '1';
77      wait for delay;
78      Addr_0 <= "0100";
79      S_0 <= '0';
80      wait for delay;
81
82      Addr_0 <= "0101";
83      S_0 <= '1';
84      wait for delay;
85      Addr_0 <= "0101";
86      S_0 <= '0';
87      wait for delay;
```

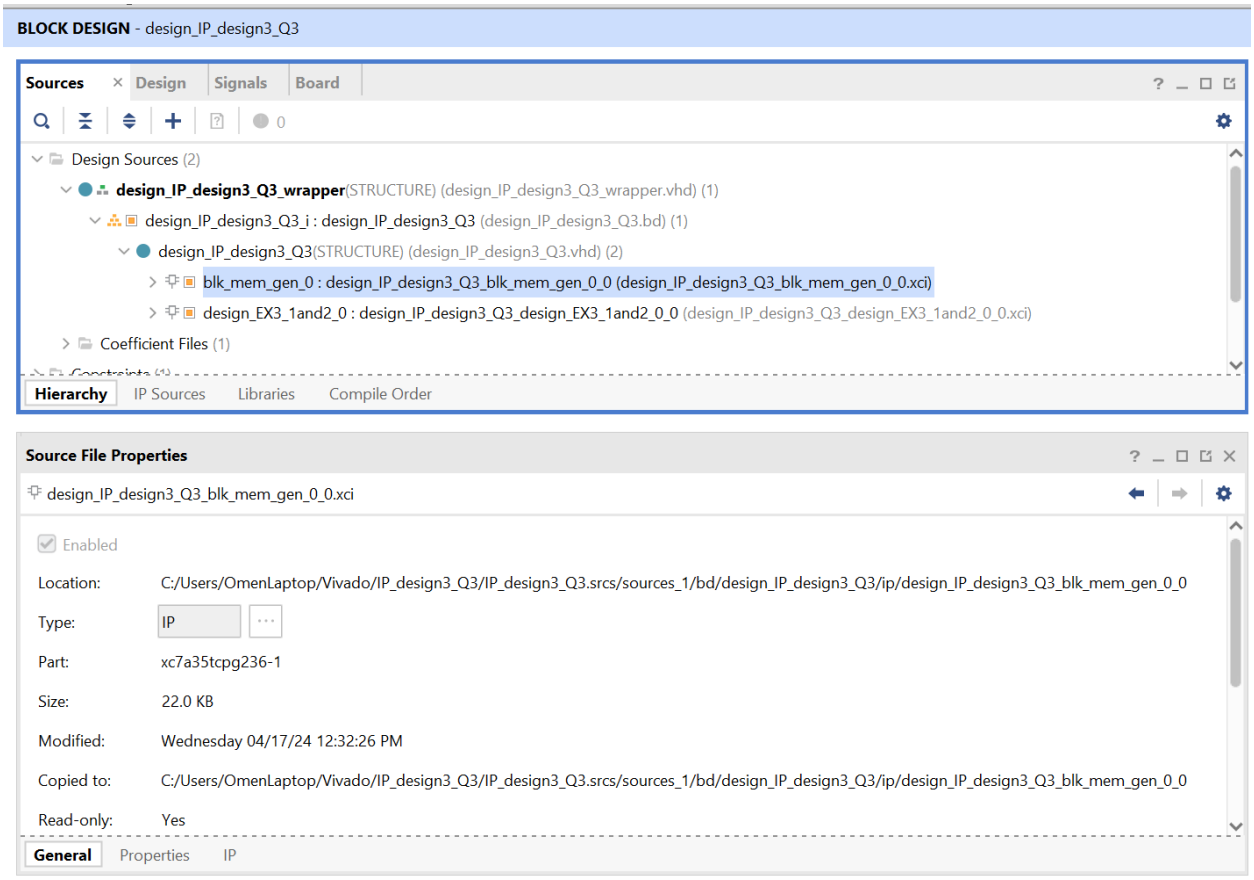
```
90      S_0 <= '1';
91      wait for delay;
92      Addr_0 <= "0110";
93      S_0 <= '0';
94      wait for delay;
95
96      Addr_0 <= "0111";
97      S_0 <= '1';
98      wait for delay;
99      Addr_0 <= "0111";
100     S_0 <= '0';
101     wait for delay;
102
103     Addr_0 <= "1000";
104     S_0 <= '1';
105     wait for delay;
106     Addr_0 <= "1000";
107     S_0 <= '0';
108     wait for delay;
109
110     Addr_0 <= "1001";
111     S_0 <= '1';
112     wait for delay;
113     Addr_0 <= "1001";
114     S_0 <= '0';
115     wait for delay;
116
117     Addr_0 <= "1010";
118     S_0 <= '1';
119     wait for delay;
120     Addr_0 <= "1010";
121     S_0 <= '0';
122     wait for delay;
123 wait;
124 end process;
125
126 end TB;
```

b) Photos/images of the project Sources and Source file properties of the project highlighting the HDL wrapper name, and location. Photos must clearly show the project wrapper source file name and saved location.

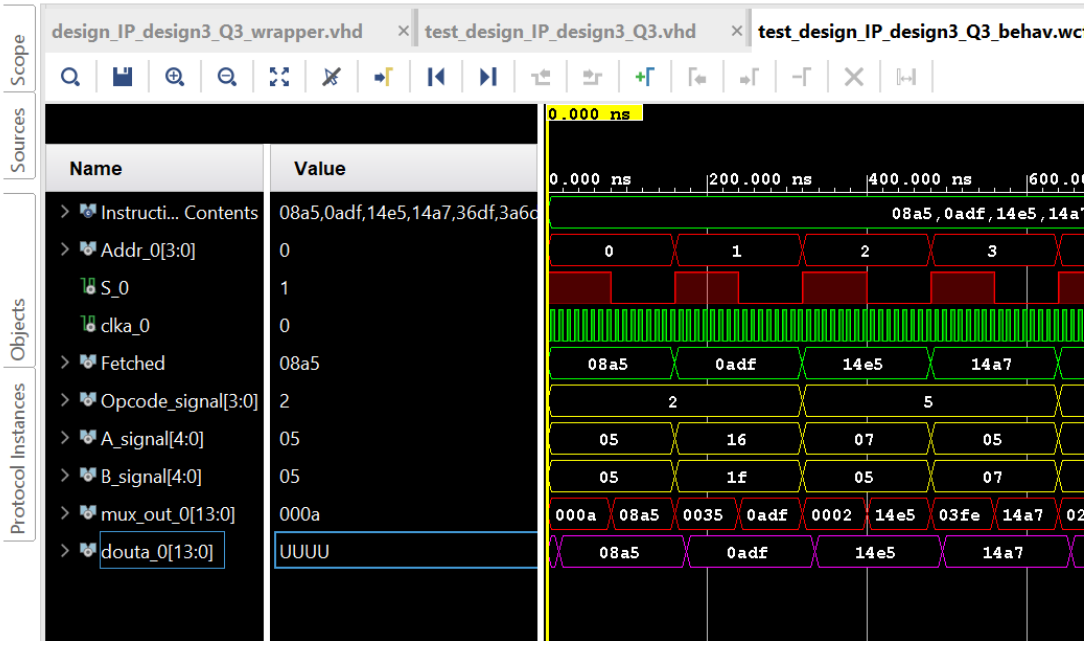


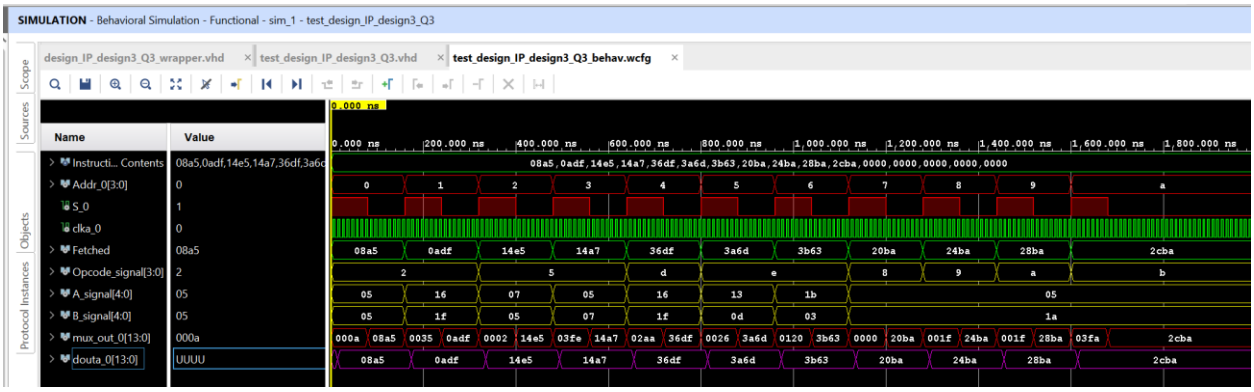
c) Photos/images of the project Sources and Source file properties of the project showing the IP library object XML file (*.xci), and the BROM generator library object. Photos must clearly show the source/object file name and saved location.



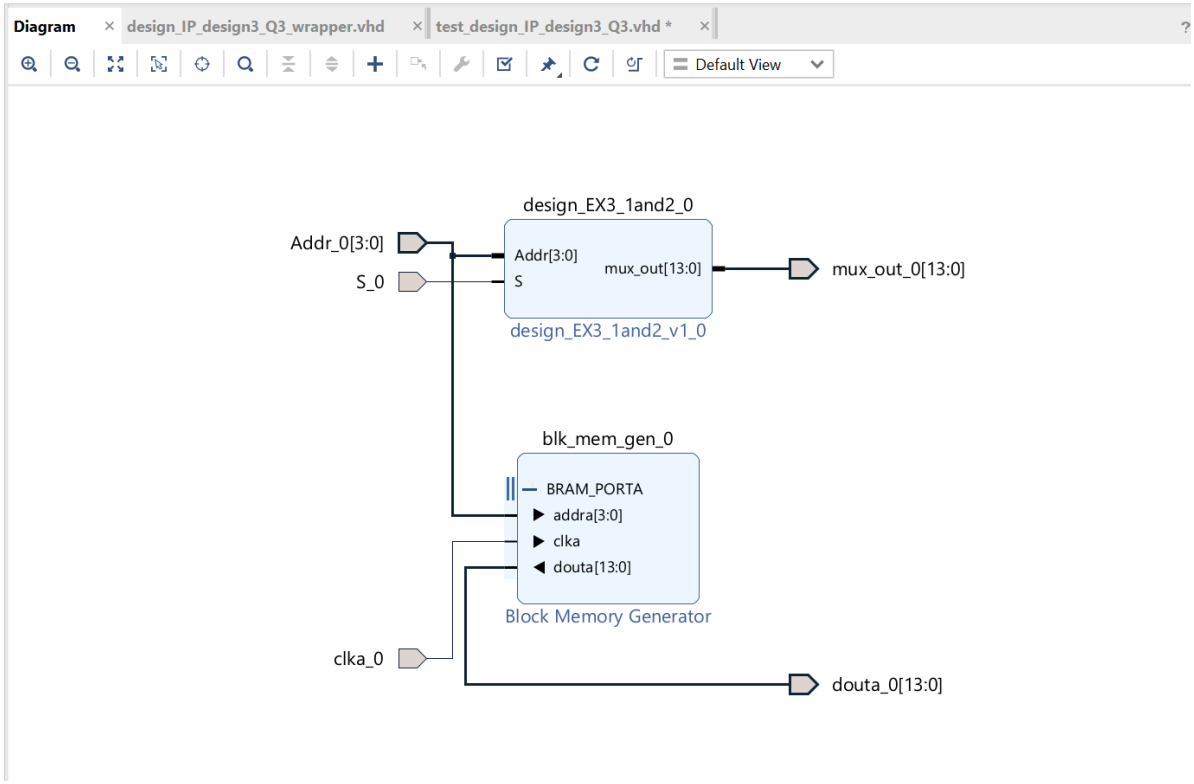


d) Photos(s)/Image(s) of the Vivado simulation timing diagram for execution of all the instructions/test stimulus inputs.

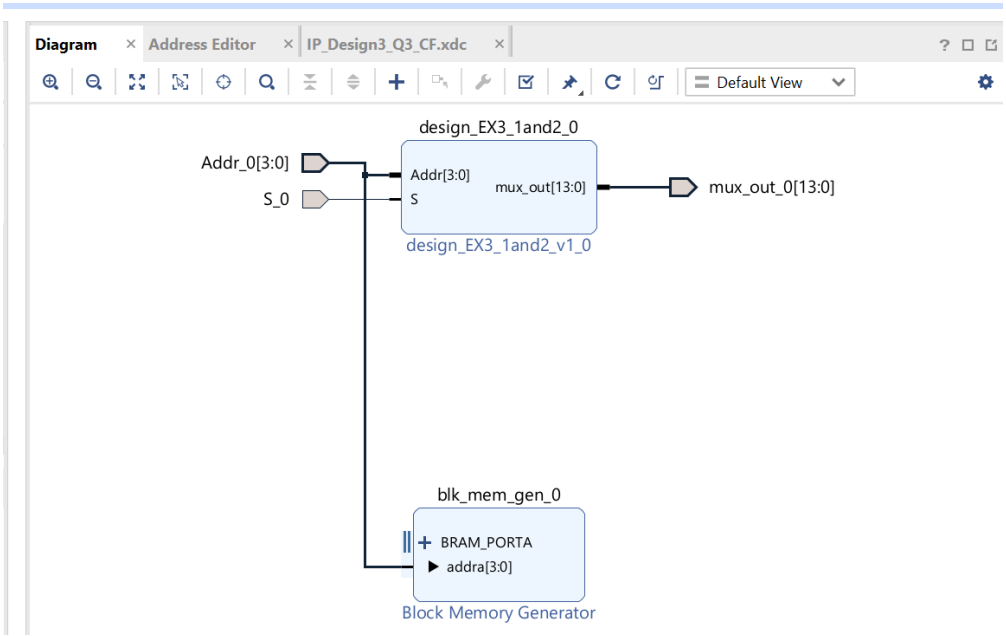



SIMULATION - Behavioral Simulation - Functional - sim_1 - test_design_IP_design3_Q3
design_IP_design3_Q3_wrapper.vhd
test_design_IP_design3_Q3.vhd
test_design_IP_design3_Q3_behav.wcfg
Scope
Sources
Objects
Protocol Instances
Name
Value
Instructi... Contents
08a5,0adf,14e5,14a7,36df,3a6d,3b63,20ba,24ba,28ba,2cba,0000,0000,0000,0000,0000
Addr_0[3:0]
S_0
clka_0
Fetched
Opcode_signal[3:0]
A_signal[4:0]
B_signal[4:0]
mux_out_0[13:0]
douta_0[13:0]
UUUU
0.000 ns
200.000 ns
400.000 ns
600.000 ns
800.000 ns
1,000.000 ns
1,200.000 ns
1,400.000 ns
1,600.000 ns
1,800.000 ns

The following Block Design is created:

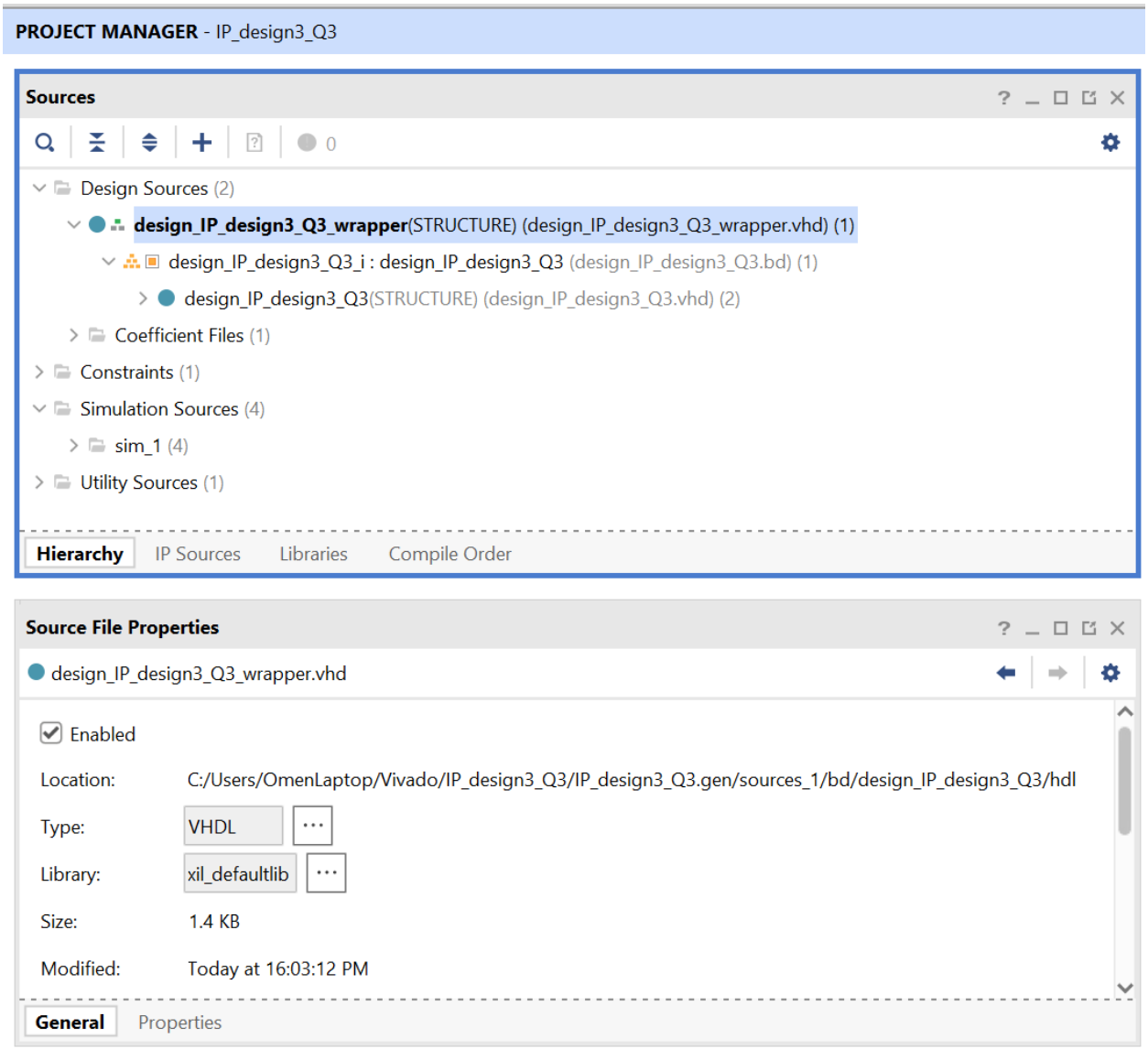


Now above Block and other parts – wrapper, TB, and others are modified according to the question i.e., NO clock and No dout:

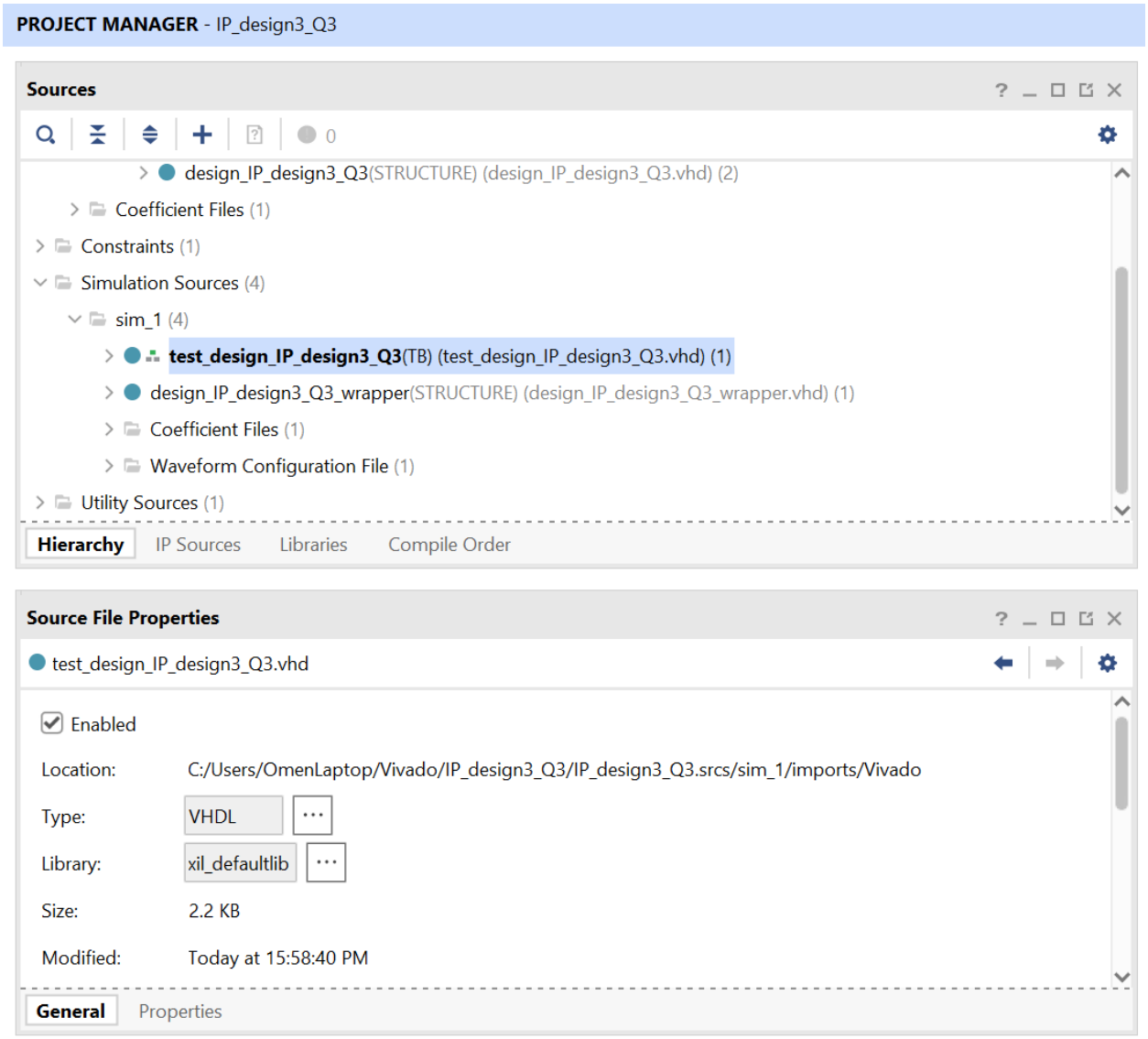


New …Wrapper after modification

```
10  ----------------------------------------------------------------------------------
11  library IEEE;
12  use IEEE.STD_LOGIC_1164.ALL;
13  library UNISIM;
14  use UNISIM.VCOMPONENTS.ALL;
15  entity design_IP_design3_Q3_wrapper is
16    port (
17      Addr_0 : in STD_LOGIC_VECTOR ( 3 downto 0 );
18      S_0 : in STD_LOGIC;
19      mux_out_0 : out STD_LOGIC_VECTOR ( 13 downto 0 )
20    );
21  end design_IP_design3_Q3_wrapper;
22
23  architecture STRUCTURE of design_IP_design3_Q3_wrapper is
24    component design_IP_design3_Q3 is
25    port (
26      Addr_0 : in STD_LOGIC_VECTOR ( 3 downto 0 );
27      S_0 : in STD_LOGIC;
28      mux_out_0 : out STD_LOGIC_VECTOR ( 13 downto 0 )
29    );
30    end component design_IP_design3_Q3;
31  begin
32  design_IP_design3_Q3_i: component design_IP_design3_Q3
33       port map (
34        Addr_0(3 downto 0) => Addr_0(3 downto 0),
35        S_0 => S_0,
36        mux_out_0(13 downto 0) => mux_out_0(13 downto 0)
37      );
38  end STRUCTURE;
39
```


PROJECT MANAGER - IP_design3_Q3
Sources
Design Sources (2)
design_IP_design3_Q3_wrapper(STRUCTURE) (design_IP_design3_Q3_wrapper.vhd) (1)
design_IP_design3_Q3_i : design_IP_design3_Q3 (design_IP_design3_Q3.bd) (1)
design_IP_design3_Q3(STRUCTURE) (design_IP_design3_Q3.vhd) (2)
Coefficient Files (1)
Constraints (1)
Simulation Sources (4)
sim_1 (4)
Utility Sources (1)
Hierarchy
IP Sources
Libraries
Compile Order
Source File Properties
design_IP_design3_Q3_wrapper.vhd
Enabled
Location:
C:/Users/OmenLaptop/Vivado/IP_design3_Q3/IP_design3_Q3.gen/sources_1/bd/design_IP_design3_Q3/hdl
Type:
VHDL
Library:
xil_defaultlib
Size:
1.4 KB
Modified:
Today at 16:03:12 PM
General
Properties

New Testbench after modification:



Project Summary | design_IP_design3_Q3_wrapper.vhd | test_design_IP_design3_Q3.vhd | IP_d

C:/Users/OmenLaptop/Vivado/IP_design3_Q3/IP_design3_Q3.srcs/sim_1/imports/Vivado/test_design_IP_design3_Q3.vhd

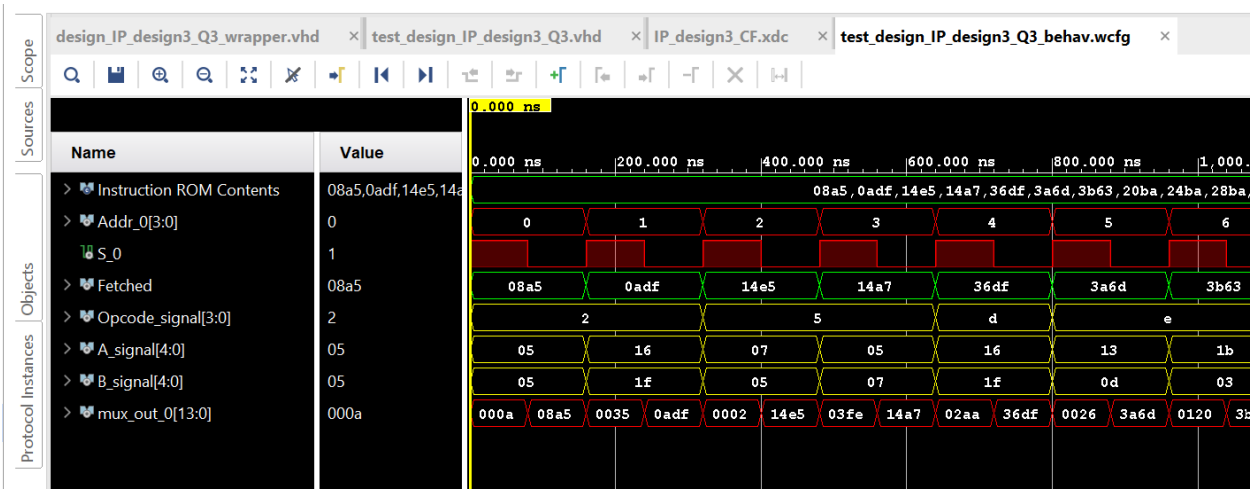
```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity test_design_IP_design3_Q3 is
5  end test_design_IP_design3_Q3;
6
7
8  architecture TB of test_design_IP_design3_Q3 is
9
10 component design_IP_design3_Q3 is
11 port (
12   Addr_0 : in STD_LOGIC_VECTOR (3 downto 0);
13   S_0 : in STD_LOGIC;
14   mux_out_0 : out STD_LOGIC_VECTOR (13 downto 0)
15 );
16 end component design_IP_design3_Q3;
17
18 signal Addr_0 : STD_LOGIC_VECTOR (3 downto 0);
19 signal S_0 : STD_LOGIC;
20 signal mux_out_0 : STD_LOGIC_VECTOR (13 downto 0);
21 constant delay : time := 80 ns;
22
23 begin
24
25 DUT: component design_IP_design3_Q3 port map (
26   Addr_0 => Addr_0,
27   S_0 => S_0,
28   mux_out_0 => mux_out_0
29 );
```

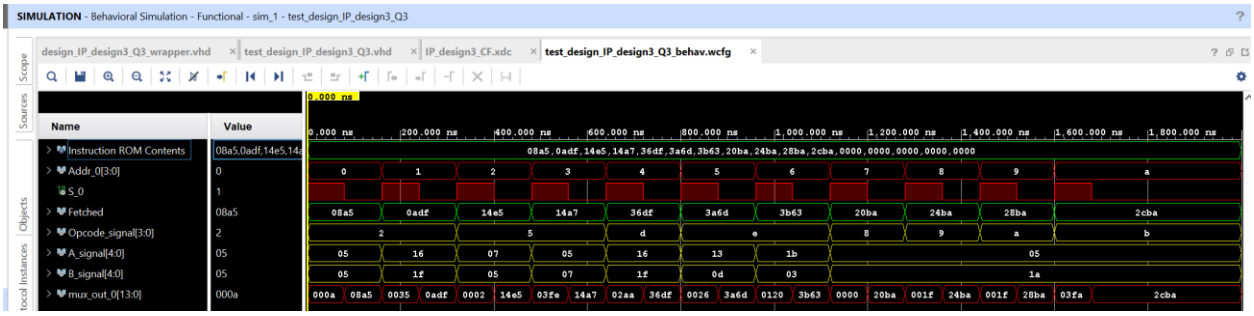
```
30 Stimulus : process
31 begin
32
33     Addr_0 <= "0000";
34     S_0 <= '1';
35     wait for delay;
36     Addr_0 <= "0000";
37     S_0 <= '0';
38     wait for delay;
39
40     Addr_0 <= "0001";
41     S_0 <= '1';
42     wait for delay;
43     Addr_0 <= "0001";
44     S_0 <= '0';
45     wait for delay;
46
47     Addr_0 <= "0010";
48     S_0 <= '1';
49     wait for delay;
50     Addr_0 <= "0010";
51     S_0 <= '0';
52     wait for delay;
53
54     Addr_0 <= "0011";
55     S_0 <= '1';
56     wait for delay;
57     Addr_0 <= "0011";
58     S_0 <= '0';
59     wait for delay;
60
61     Addr_0 <= "0100";
62     S_0 <= '1';
63     wait for delay;
64     Addr_0 <= "0100";
65     S_0 <= '0';
66     wait for delay;
```

```
    Addr_0 <= "0101";
    S_0 <= '1';
    wait for delay;
    Addr_0 <= "0101";
    S_0 <= '0';
    wait for delay;

    Addr_0 <= "0110";
    S_0 <= '1';
    wait for delay;
    Addr_0 <= "0110";
    S_0 <= '0';
    wait for delay;

    Addr_0 <= "0111";
    S_0 <= '1';
    wait for delay;
    Addr_0 <= "0111";
    S_0 <= '0';
    wait for delay;

    Addr_0 <= "1000";
    S_0 <= '1';
    wait for delay;
    Addr_0 <= "1000";
    S_0 <= '0';
    wait for delay;

    Addr_0 <= "1001";
    S_0 <= '1';
    wait for delay;
    Addr_0 <= "1001";
    S_0 <= '0';
    wait for delay;
```

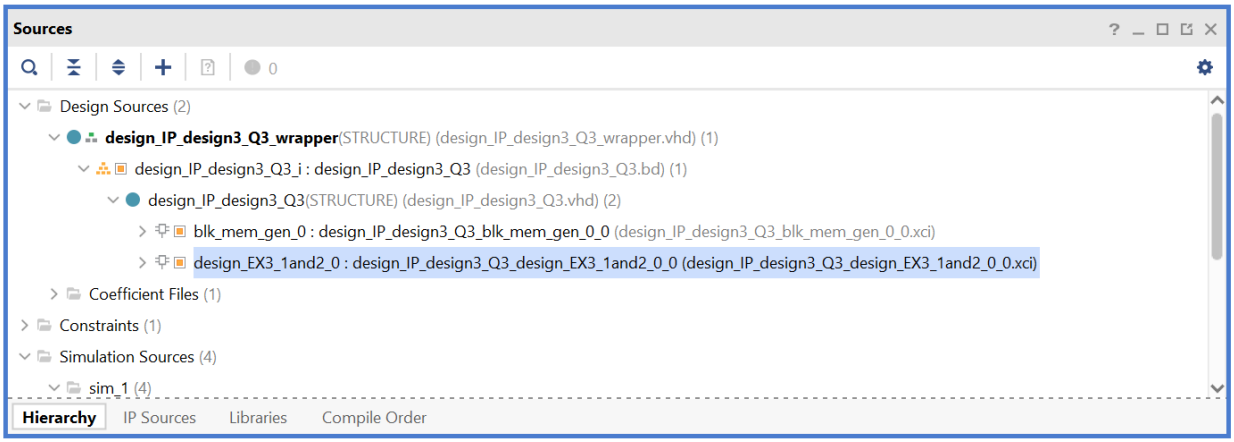
```
    wait for delay;

    Addr_0 <= "1010";
    S_0 <= '1';
    wait for delay;
    Addr_0 <= "1010";
    S_0 <= '0';
    wait for delay;

wait;
end process;

end TB;
```

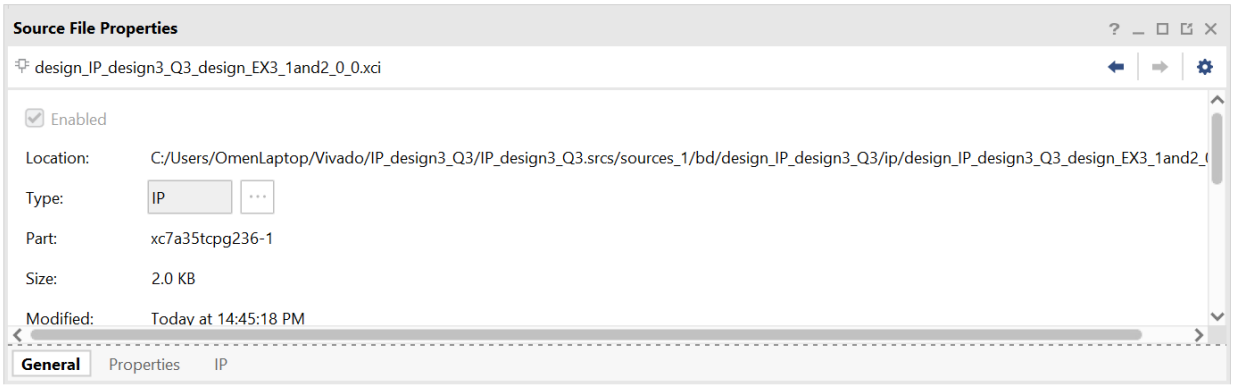
New Stimulation after modification:

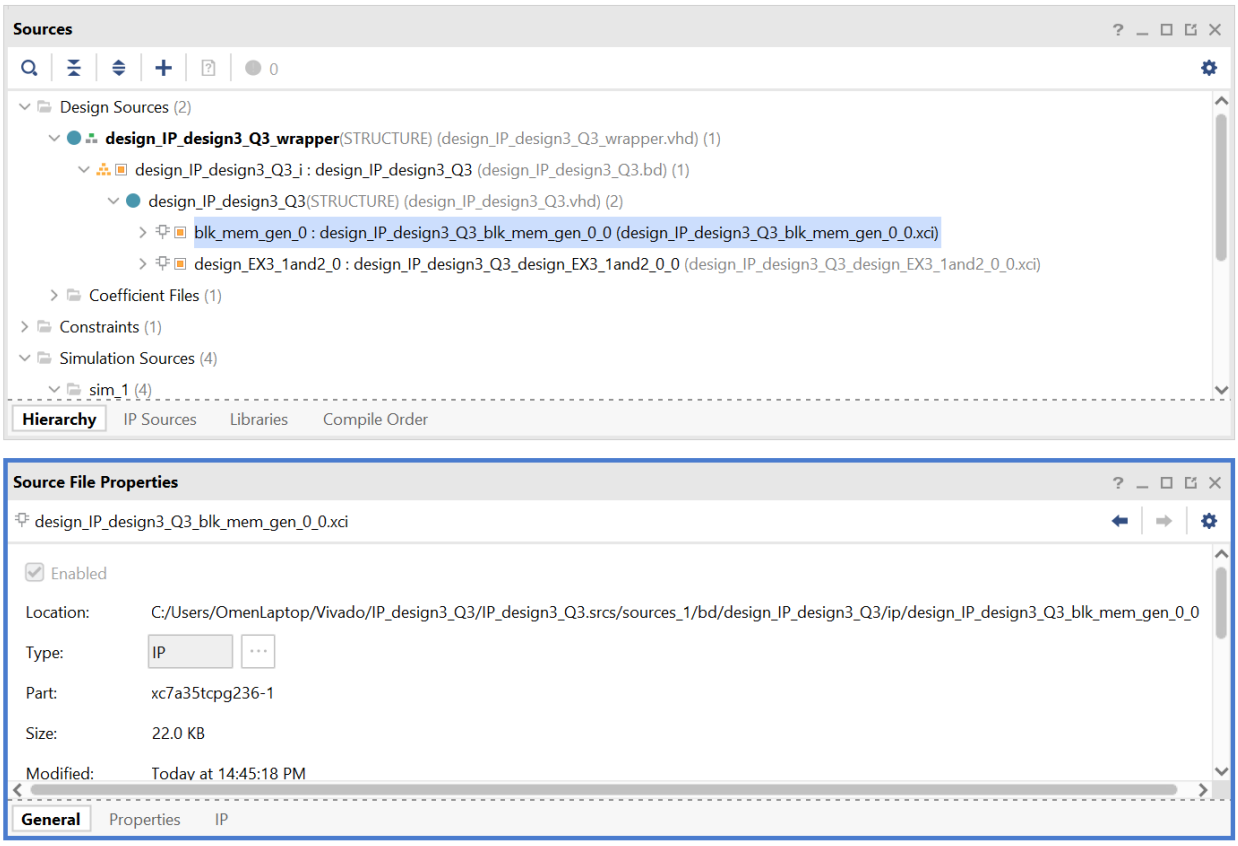




For .xci File after modification:



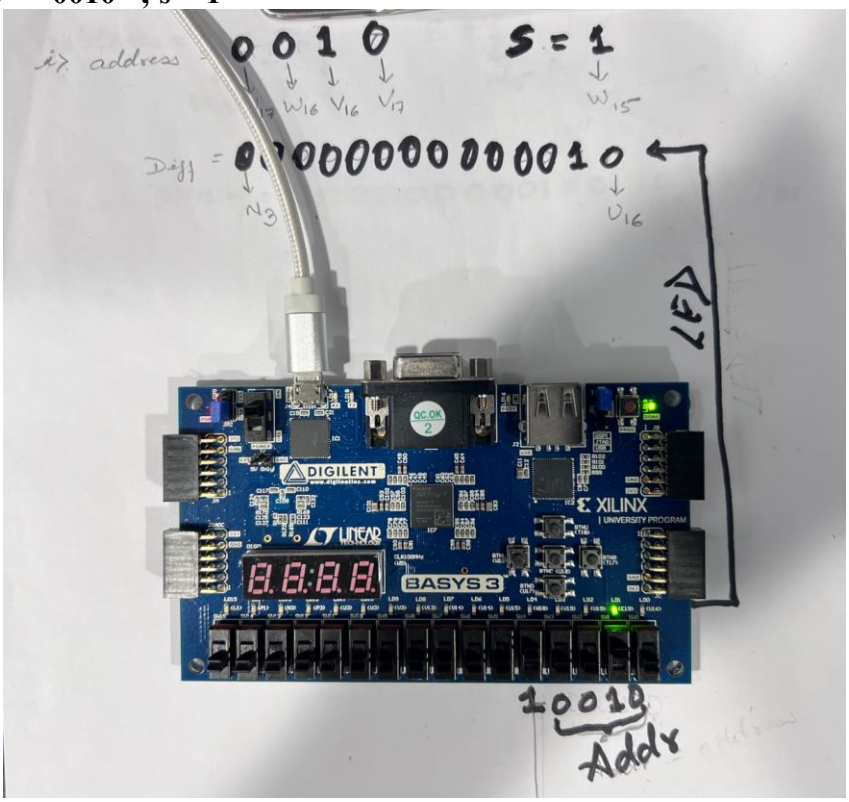




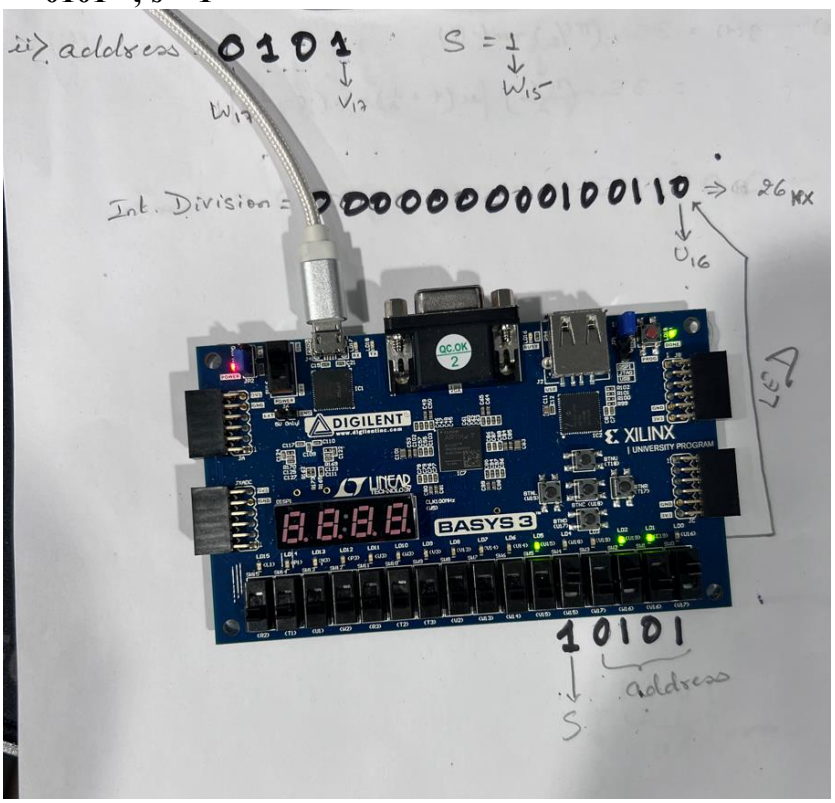
e) Photos/Images(s) of the FPGA board running your design

demonstrating the correct operation of the following selected examples.

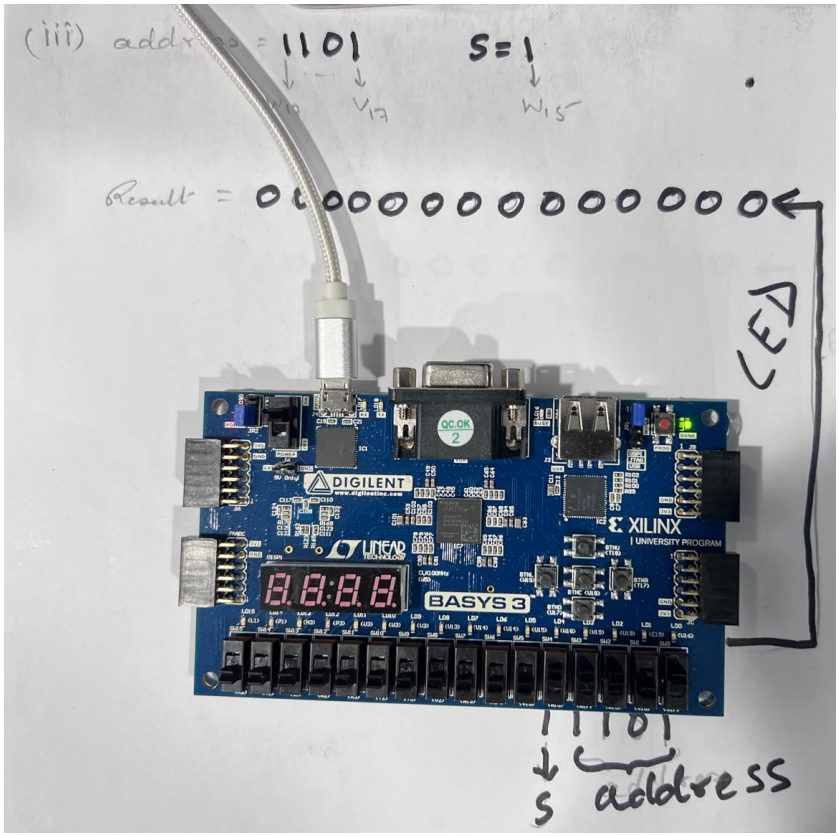
I. **address = “0010” ; s = 1**



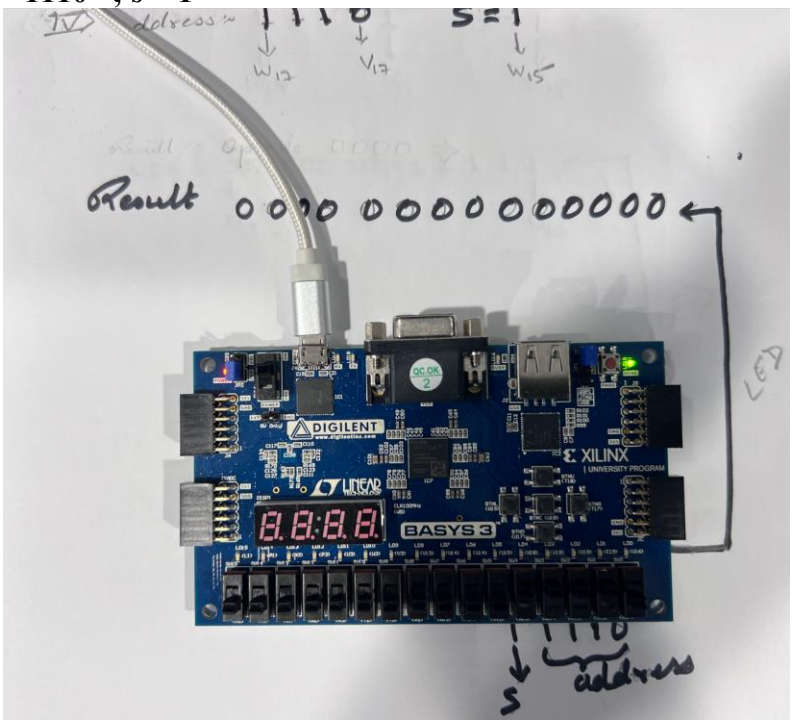
II. **address = “0101” ; s = 1**



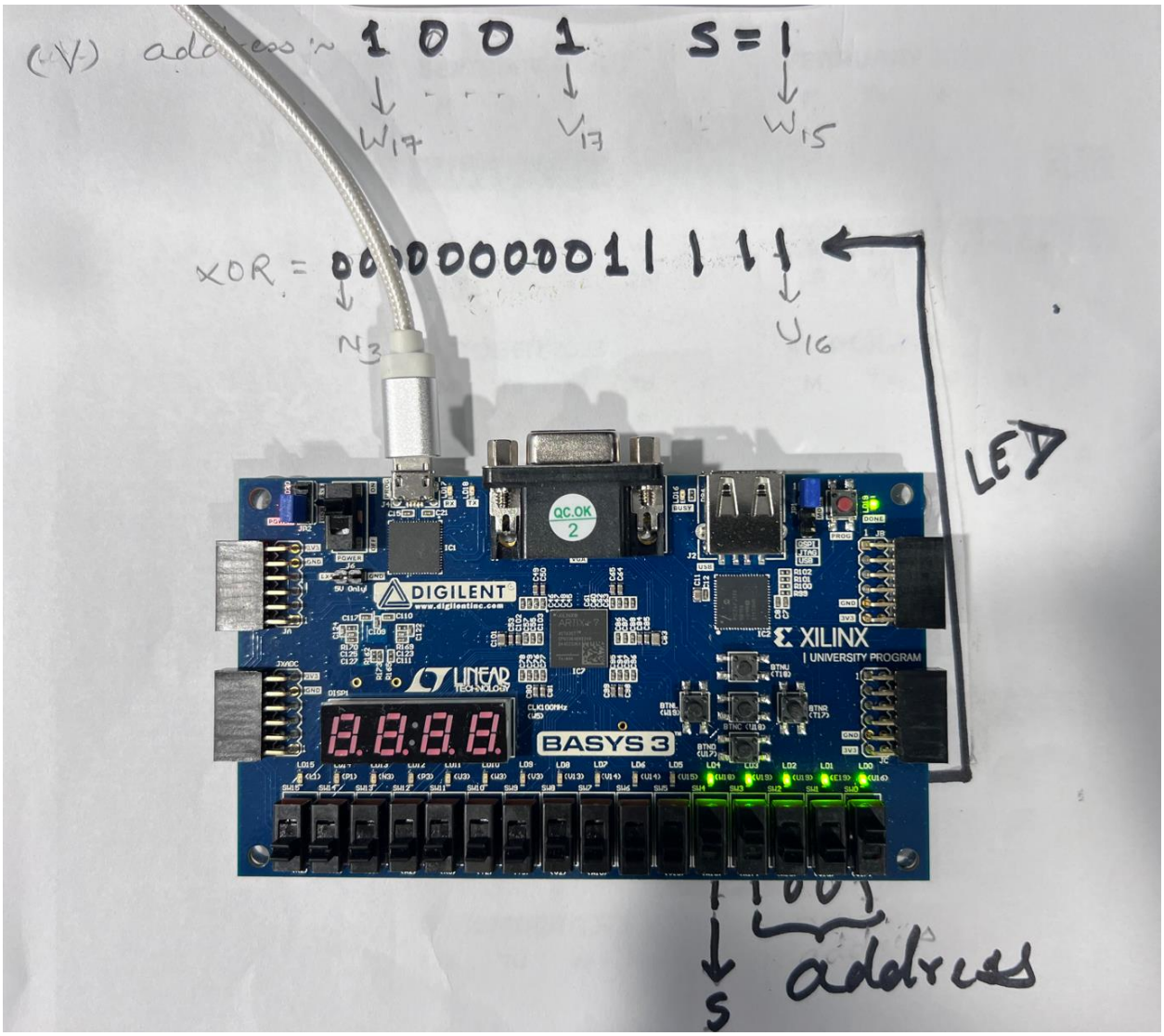
III. address = “1101” ; s = 1



IV. address = “1110” ; s = 1



V. address = “1001” ; s = 1



Conclusion: The expected result was received from both i.e., when the given signals pass through from ROM to ALU and through the IP catalog object (IPIBD),